
TRÍ TUỆ NHÂN TẠO

Tập dữ liệu, khái quát hóa và điều chỉnh quá mức

Lý thuyết — Chất lượng dữ liệu, chia tập, L2, loss curves

Biên soạn:

ĐẶNG TẤN QUÍ

SĐT: 0377 122 210

2026

Mục lục

Lời nói đầu	3
1 Giới thiệu	4
2 Đặc điểm dữ liệu (Data characteristics)	4
2.1 Các loại dữ liệu	4
2.2 Số lượng dữ liệu	5
2.3 Chất lượng và độ tin cậy (quality & reliability)	5
2.4 Example đầy đủ và không đầy đủ	6
3 Nhãn (Labels)	8
3.1 Dữ liệu do con người tạo (human-generated)	8
4 Tập dữ liệu mất cân bằng lớp (Class-imbalanced datasets)	9
4.1 Balanced vs imbalanced	9
4.2 Vì sao train imbalanced nghiêm trọng khó?	10
4.3 Train dataset imbalanced: downsampling + upweighting	10
5 Chia tập dữ liệu ban đầu (Dividing the original dataset)	11
5.1 Training set, validation set, test set	11
5.2 Vấn đề bổ sung của test set	12
6 Chuyển đổi dữ liệu (Transforming data)	13
6.1 Sample khi dữ liệu quá nhiều	14
6.2 Lọc PII	14
7 Tổng quát hóa (Generalization)	14
8 Quá trình điều chỉnh quá mức (Overfitting)	14
8.1 Ví dụ: rừng cây khỏe / cây bệnh	14
8.2 Underfit, fit và overfit	15
8.3 Phát hiện overfitting	15
8.4 Nguyên nhân overfitting	16
8.5 Điều kiện tổng quát hóa (điều kiện dataset)	16
9 Độ phức tạp của mô hình (Model complexity) và Regularization	17
9.1 Regularization	18
9.2 Loss và complexity	18
9.3 Complexity đo bằng gì?	18
10 Điều chỉnh L2 (L_2 regularization) và lambda	19
10.1 Regularization rate (lambda)	20
10.2 Chọn lambda	20
10.3 Early stopping: lựa chọn thay thế	20
10.4 Cân bằng learning rate và regularization rate	21

11 Diễn giải đường cong tổn thất (Interpreting loss curves)	21
11.1 Bài tập 1: loss dao động (oscillating)	21
11.2 Bài tập 2: loss nhảy vọt (sharp jump)	22
11.3 Bài tập 3: validation loss tăng còn training loss giảm	22
11.4 Bài tập 4: loss bị kẹt / dạng sóng chữ nhật	23
12 Tổng kết	23

Đặng Tấn Quý

Lời nói đầu

Nguồn và cách đọc

Tài liệu biên soạn và dịch đầy đủ từ *Datasets, Generalization, and Overfitting* (Google Machine Learning Crash Course)^a, tương ứng **11 phần** (file 1–11) trong thư mục. Mục tiêu là giữ nội dung cốt lõi, các câu hỏi trắc nghiệm “check your intuition/understanding” và các gợi ý đáp án, tránh tóm tắt quá mức.

^a<https://developers.google.com/machine-learning/crash-course/overfitting>

Thời lượng ước tính

Khoảng **105 phút** theo MLCC.

Mục tiêu học tập

- Nhận diện bốn đặc điểm của dữ liệu và dataset.
- Nêu ít nhất bốn nguyên nhân khiến dữ liệu kém tin cậy.
- Biết khi nào nên xóa dữ liệu thiếu và khi nào nên impute.
- Phân biệt nhãn trực tiếp và nhãn proxy/được suy ra.
- Nêu hai cách cải thiện chất lượng nhãn do con người gán.
- Giải thích vì sao cần chia dataset thành train/validation/test và một lỗi thường gặp khi chia tập.
- Giải thích overfitting và ít nhất ba nguyên nhân.
- Giải thích regularization, đặc biệt L2 (lambda) và early stopping.
- Đọc các dạng loss curve: hội tụ, lỗi học, và dấu hiệu overfitting.

Điều kiện tiên quyết

Giả định bạn đã học:

- [Introduction to Machine Learning](#)
- [Linear regression](#)
- [Working with numerical data](#)
- [Working with categorical data](#)

1 Giới thiệu

Câu hỏi dẫn dắt: cải thiện thứ gì có tác động nhất?

Nếu buộc phải ưu tiên cải thiện **một** thứ trong dự án ML, cái nào tác động nhất?

- **Cải thiện chất lượng dataset.** (Dữ liệu “ăn” tất cả: chất lượng và quy mô dataset quan trọng hơn nhiều so với thuật toán “hào nhoáng”.)
- Dùng một hàm mất mát “thông minh” hơn. (Có thể giúp train nhanh hơn, nhưng vẫn đứng sau chất lượng dữ liệu.)

Câu hỏi dẫn dắt thứ hai: thời gian làm dữ liệu

dự án ML, bạn thường dành bao nhiêu thời gian cho chuẩn bị và biến đổi dữ liệu?

- **Hơn một nửa thời gian.** (Đúng: đa số thời gian là dựng dataset và feature engineering.)
- Ít hơn một nửa. (Thực tế, thường khoảng **80%** thời gian của dự án ML nằm ở dựng dataset và biến đổi dữ liệu.)

Ý nghĩa & Trực giác

Trong module này, ta học đặc điểm của dataset ML và cách chuẩn bị dữ liệu để đạt kết quả chất lượng cao khi huấn luyện và đánh giá mô hình.

2 Đặc điểm dữ liệu (Data characteristics)

Dataset và example

Dataset là tập các **example**. Nhiều dataset lưu dạng bảng (CSV, spreadsheet, bảng DB): mỗi hàng là một example, mỗi cột là feature hoặc label tiềm năng. Tuy nhiên dataset cũng có thể xuất phát từ log, protocol buffers, v.v.

Động cơ học – Tại sao cần khái niệm này?

ô hình ML chỉ tốt bằng dữ liệu mà nó được train. Phần này xem các đặc điểm quan trọng của dữ liệu.

2.1 Các loại dữ liệu

Dataset có thể chứa nhiều kiểu dữ liệu, ví dụ:

- Dữ liệu số (xem module Numerical Data)
- Dữ liệu danh mục (xem module Categorical Data)
- Ngôn ngữ tự nhiên (từ, câu, cả tài liệu)

- Đa phương tiện (ảnh, video, âm thanh)
- Output từ hệ thống ML khác
- [Embedding vectors](#) (xem unit sau)

2.2 Số lượng dữ liệu

Quy tắc ngón tay cái

: nên có số example ít nhất lớn hơn số tham số trainable **1–2 bậc độ lớn** (10x đến 100x). Thực tế, mô hình tốt thường cần nhiều hơn thế.

Lưu ý

Mô hình train trên dataset lớn với ít feature thường vượt mô hình train trên dataset nhỏ với nhiều feature. Google từng thành công lớn với mô hình đơn giản trên dataset lớn. vậy, lượng dữ liệu cần thiết phụ thuộc bài toán: có bài chỉ cần vài chục ví dụ; có bài cả nghìn tỷ cũng chưa đủ. ài ra, nếu bạn đang **thích nghi** một mô hình đã train trên rất nhiều dữ liệu cùng schema (transfer/adaptation), dataset nhỏ vẫn có thể cho kết quả tốt.

2.3 Chất lượng và độ tin cậy (quality & reliability)

Định nghĩa thực dụng về chất lượng

khóa này, **dataset chất lượng cao** là dataset giúp mô hình đạt mục tiêu; dataset chất lượng thấp là dataset cản trở mô hình đạt mục tiêu.

Reliability

Reliability là mức độ bạn có thể **tin** dữ liệu. Mô hình train trên dữ liệu tin cậy có khả năng dự đoán hữu ích hơn.

Đo reliability: cần tự hỏi

- Lỗi nhân phổ biến đến đâu? (Nếu nhân do người gán, họ sai bao nhiêu lần?)
- Feature có nhiều không? (Giá trị feature có lỗi đo đạc; không thể loại sạch mọi nhiễu; GPS dao động nhẹ là bình thường.)
- Dữ liệu đã lọc đúng cho bài toán chưa? Ví dụ: nên gồm truy vấn từ bot không? (Spam detection: có thể nên; tối ưu search cho người: không.)

Nguyên nhân phổ biến làm dữ liệu kém tin cậy

- Thiếu giá trị (omitted values)
- Ví dụ trùng lặp (duplicate examples)
- Giá trị feature lỗi (gõ thừa số, nhiệt kế phơi nắng)

- Nhãn sai (gán nhầm ảnh cây sồi thành cây phong)
- Một đoạn dữ liệu xấu (một ngày mạng chập chờn làm feature sai hàng loạt)

ên nghị dùng tự động hóa để gán cờ dữ liệu không tin cậy: unit tests dựa trên data schema để phát hiện giá trị vượt miền.

Lưu ý

đủ lớn/đa dạng gần như chắc chắn có **outliers** vượt schema hoặc dải unit test. Xử lý outlier là phần quan trọng (xem module Numerical Data).

2.4 Example đầy đủ và không đầy đủ

Complete vs incomplete

đầy đủ có đủ giá trị cho mọi feature. Thực tế, nhiều example **không đầy đủ** (thiếu ít nhất một feature).

Thiếu hình minh họa

Không tìm thấy file `complete_example.png` trong `thm/images/overfitting/`.

Hình 1. Một example đầy đủ (đủ 5 feature).

Thiếu hình minh họa

Không tìm thấy file `incomplete_example.png` trong `thm/images/overfitting/`.

Hình 2. Một example thiếu một feature.

Lưu ý

ông nên train trên example thiếu. Hãy sửa bằng một trong hai cách:

- Xóa các example thiếu.
- **Impute** (điền) giá trị thiếu bằng ước lượng hợp lý.

Thiếu hình minh họa

Không tìm thấy file `delete_incomplete_examples.png` trong `thm/images/overfitting/`.

Hình 3. Xóa các example không đầy đủ.

Thiếu hình minh họa

Không tìm thấy file `imputemissingvalues.png` trong `thmimages/overfitting/`.

Hình 4. Impute để biến example thiếu thành đầy đủ.

Khi nào xóa? khi nào impute?

Để dataset có đủ example đầy đủ để train mô hình hữu ích, cân nhắc xóa các example thiếu. Nếu chỉ một feature bị thiếu nhiều và feature đó có thể không giúp nhiều, cân nhắc bỏ feature đó khỏi input rồi so sánh chất lượng. Nếu không đủ example đầy đủ, cân nhắc impute. Ví dụ example vô dụng thì ổn, nhưng xóa example quan trọng là nguy hiểm. Nếu khó quyết, hãy dựng hai dataset: một cái xóa, một cái impute, rồi xem cái nào train tốt hơn.

Lưu ý về imputation

Việc toán impute có thể thông minh, nhưng giá trị impute hiếm khi tốt bằng giá trị thật. Dataset tốt nên cho mô hình biết giá trị nào là impute bằng cách thêm cột Boolean, ví dụ

`temperature`

`is`

`imputed`. Khi `imputed` là "tin" thì `impute` là giá trị thật, ngược lại là giá trị impute. Có lý do, không phải ngẫu nhiên là impute.

Bài tập: nhiệt độ thiếu 11:00

Cảnh báo: dataset đã sort có thể giúp impute, nhưng **không** nên train trên dataset đã sort. Sau khi impute, hãy shuffle. Ảnh theo thời gian:

Timestamp	Temperature
8, 2023 09:00	12
8, 2023 10:00	18
8, 2023 11:00	missing
8, 2023 12:00	24
8, 2023 13:00	38

Giá trị nào hợp lý để impute?

- **23: Có thể.** 23 là mean của các giá trị lân cận (12, 18, 24, 38). Tuy nhiên cần xem toàn dataset để chắc nó không là outlier vào 11:00 các ngày khác.
- **31:** Ít khả năng; nhìn đoạn dữ liệu này thì 31 quá cao, nhưng cần nhiều ví dụ hơn để chắc.

- 51: Rất ít khả năng; lớn hơn mọi giá trị hiển thị.

3 Nhãn (Labels)

Nhãn trực tiếp và nhãn proxy

loại nhãn:

- **Direct labels:** nhãn trùng đúng thứ mô hình cần dự đoán (cột đó có sẵn trong dataset). Ví dụ cột `textttbicycle owner` là direct label cho bài toán dự đoán có sở hữu xe đạp hay không.
- **Proxy labels:** nhãn gần giống nhưng không trùng hoàn toàn với mục tiêu. Ví dụ người đăng ký tạp chí xe đạp có thể (nhưng không chắc) sở hữu xe đạp.

label thường tốt hơn proxy. Nếu có direct label thì nên dùng. Nhưng đôi khi direct label không tồn tại. label luôn là thỏa hiệp: xấp xỉ không hoàn hảo. Proxy chỉ hữu ích nếu mối liên hệ giữa proxy và mục tiêu đủ mạnh.

Lưu ý

ớ rằng mọi label cuối cùng đều phải được biểu diễn dưới dạng số thực (tương tự feature vector), vì ML rốt cuộc là các phép toán. Đôi khi direct label có nhưng khó biểu diễn float; khi đó dùng proxy.

Bài tập: gửi coupon cho chủ xe đạp

ục tiêu: dự đoán ai sở hữu xe đạp. Dataset không có cột `textttbike owner` nhưng có cột

`textttrecently bought a bicycle`. Đây là proxy tốt hay kém? **Đáp án: Proxy tương đối tốt.** Người mua xe đạp thường sở hữu xe đạp, nhưng vẫn không hoàn hảo (mua làm quà, v.v.).

3.1 Dữ liệu do con người tạo (human-generated)

Ý nghĩa & Trực giác

ột số dữ liệu được tạo bởi con người: người xem thông tin và cung cấp giá trị (thường là nhãn). Ví dụ nhà khí tượng nhìn ảnh mây để nhận dạng loại mây. ột số dữ liệu được tạo tự động (software, có thể là mô hình ML khác) gán nhãn.

Ưu và nhược của nhãn do con người

Ưu điểm:

- Con người làm được nhiều nhiệm vụ mà mô hình ML dù tinh vi vẫn khó.
- Buộc chủ dataset phải xây tiêu chí rõ ràng và nhất quán.

Nhược điểm:

- Tốn chi phí trả người gán nhãn.
- Con người có thể sai, nên cần nhiều người đánh giá cùng dữ liệu.

Cần suy nghĩ để quyết định nhu cầu

- Rater cần kỹ năng gì? (biết ngôn ngữ cụ thể? cần nhà ngôn ngữ học?)
- Cần bao nhiêu mẫu đã gán nhãn và cần nhanh thế nào?
- Ngân sách?

Luôn double-check người gán nhãn

Ấy tự gán nhãn khoảng 1000 ví dụ và so sánh với rater khác. Nếu có sai khác, đừng mặc định bạn đúng (nhất là khi có yếu tố đánh giá chủ quan). Nếu rater mắc lỗi, bổ sung hướng dẫn và thử lại.

Ý nghĩa & Trực giác

Ìn dữ liệu bằng tay là bài tập tốt dù bạn lấy dữ liệu bằng cách nào. Andrej Karpathy từng làm điều này với ImageNet và viết lại trải nghiệm. ô hình có thể train trên hỗn hợp nhãn tự động và nhãn người, nhưng với đa số mô hình, thêm một bộ nhãn người (dễ lỗi thời) thường không đáng với độ phức tạp bảo trì. Tuy nhiên, đôi khi nhãn người có thông tin bổ sung mà nhãn tự động không có.

4 Tập dữ liệu mất cân bằng lớp (Class-imbalanced datasets)

Ý nghĩa & Trực giác

ần này trả lời:

- Balanced vs imbalanced khác nhau thế nào?
- Vì sao train imbalanced khó?
- Làm sao vượt qua?

4.1 Balanced vs imbalanced

Balanced / Imbalanced

ét nhãn danh mục chỉ có positive/negative. Dataset **cân bằng** khi số positive và negative xấp xỉ nhau. Dataset **mất cân bằng** khi một nhãn phổ biến hơn hẳn. dataset imbalanced:

- Nhãn phổ biến gọi là **majority class**
- Nhãn hiếm gọi là **minority class**

ài đời, imbalanced phổ biến hơn balanced: gian lận thẻ <0,1%, virus hiếm <0,01%.

4.2 Vì sao train imbalanced nghiêm trọng khó?

Ý nghĩa & Trực giác

ấn luyện cần batch có đủ ví dụ của cả hai lớp. Với imbalanced nhẹ, batch nhỏ vẫn có đủ. Với imbalanced nặng, có thể không đủ ví dụ minority để train.

Thiếu hình minh họa

Không tìm thấy file `FloralDataset200Sunflowers2Roses.png` trong thư mục `images/overfitting/`.

Hình 6. Dataset lệch mạnh: 200 sunflower, 2 rose.

Lưu ý

ều batch size = 20, đa số batch không có minority. Batch size = 100 thì trung bình 1 minority/batch, vẫn không đủ. Batch lớn hơn cũng vẫn lệch. ưu ý: [Accuracy](#) thường là metric tệ cho dataset imbalanced.

4.3 Train dataset imbalanced: downsampling + upweighting

Ý nghĩa & Trực giác

training, mô hình cần học hai thứ:

- Mỗi lớp “trông như thế nào” (liên hệ feature $\rightarrow$ class)
- Mỗi lớp phổ biến ra sao (phân phối class)

chuẩn trộn lẫn hai mục tiêu này. Kỹ thuật **downsampling + upweighting majority class** tách chúng ra để mô hình đạt cả hai.

Bước 1: Downsample majority class

[Downsampling](#) là train trên tỉ lệ nhỏ bất thường của majority class: bỏ nhiều ví dụ majority để dataset “cân bằng hơn” một cách nhân tạo. Điều này tăng xác suất mỗi batch có đủ minority để train hiệu quả. í dụ, dataset 99% majority và 1% minority. Downsample majority theo hệ số 25 tạo train set cân bằng hơn (80% majority, 20% minority):

Thiếu hình minh họa

Không tìm thấy file `FloralDatasetDownsampling.png` trong thư mục `images/overfitting/`.

Hình 7. Downsampling majority theo hệ số 25.**Bước 2: Upweight lớp đã downsample**

tạo **prediction bias** vì mô hình thấy “thế giới giả” cân bằng hơn thực tế. Để sửa, hãy **upweight** majority class bằng đúng hệ số downsample: nếu downsample 25 lần thì khi dự đoán sai majority, nhân loss lên 25 (coi như 25 lỗi).

Thiếu hình minh họa

Không tìm thấy file `FloralDatasetUpweighting.png` trong thư mục `images/overfitting/`.

Hình 8. Upweight majority theo hệ số 25.**Lưu ý**

/upweight bao nhiêu? Hãy thử như hyperparameter.ợi ích:

- Mô hình vừa học được liên hệ feature-label, vừa “biết” phân phối class thật.
- Hội tụ nhanh hơn vì minority xuất hiện thường hơn.

5 Chia tập dữ liệu ban đầu (Dividing the original dataset)

Ý nghĩa & Trực giác

ũng như dự án phần mềm cần test, mô hình ML cũng cần được kiểm thử trên dữ liệu khác với dữ liệu huấn luyện.

5.1 Training set, validation set, test set

Lưu ý

ạn nên kiểm thử mô hình trên một tập ví dụ **khác** tập train. Cách truyền thống: chia dataset gốc.

Chia 2 tập: train/test (ý tưởng ổn nhưng chưa tối ưu)

ạn có thể nghĩ chia thành:

- **Training set** để train.
- **Test set** để đánh giá.

MLCC, hình minh họa thường là khoảng 80% train, 20% test.

Bài tập: nhiều vòng tối ưu dựa trên test set có vấn đề gì?

ều bạn lặp nhiều vòng: train trên train set, đánh giá trên test set, rồi dùng kết quả test để chỉnh hyperparameter/feature set; điều gì sai?**Đáp án:** làm như vậy khiến mô hình dần **khớp theo đặc thù** của test set (giống “dạy theo đề”), làm giảm khả năng tổng quát ra dữ liệu thực.

Ý nghĩa & Trực giác

2 tập là ý tưởng ổn, nhưng tốt hơn là chia 3 tập: thêm validation set.

Thiếu hình minh họa

Không tìm thấy file PartitionThreeSets.png trong thư mục images/overfitting/.

Hình 9. Chia 70% train, 15% validation, 15% test (ví dụ điển hình).

Vai trò của validation

ùng **validation set** để đánh giá trong quá trình phát triển và điều chỉnh mô hình. Khi dùng validation nhiều lần và thấy mô hình ổn, hãy dùng test set để **xác nhận cuối**.

Lưu ý về transform

bạn transform một feature trong training set, bạn phải áp dụng **y hệt** transform đó cho validation set, test set và dữ liệu thực.

Ý nghĩa & Trực giác

cả với quy trình đúng, test/validation cũng có thể “mòn” vì dùng lặp lại: càng dùng một tập dữ liệu để ra quyết định, càng kém tự tin mô hình sẽ tốt trên dữ liệu mới. Cách xử lý: thu thập thêm dữ liệu để “làm mới” test/validation.

Bài tập: test loss thấp bất thường, có thể sai gì?

ạn đã shuffle và chia train/val/test, nhưng test loss thấp đến mức nghi ngờ. Có thể sai gì?**Đáp án:** test set có nhiều ví dụ trùng với training set. Dataset có thể có ví dụ dư thừa; nên xóa trùng khỏi test set trước khi test.

5.2 Vấn đề bổ sung của test set

Lưu ý

khi chia, hãy xóa mọi ví dụ trong validation/test mà bị trùng với training: test công bằng phải là trên ví dụ **mới**, không phải bản sao.

Ví dụ spam email

ạn chia 80-20 train/test, train xong đạt 99% precision cả train và test. Hóa ra nhiều ví dụ trong test trùng với train vì bạn không scrub duplicate entries trước khi chia. Bạn đã vô tình train trên một phần test.

Một test/validation set tốt

- Đủ lớn để kết quả có ý nghĩa thống kê.
- Đại diện cho dataset nói chung (không khác tính chất với train).
- Đại diện cho dữ liệu thực mà mô hình sẽ gặp.
- Không có ví dụ trùng với training set.

Bài tập: phát biểu đúng

- **Mỗi ví dụ dùng để test là một ví dụ không được dùng để train.** (Đúng: chia train/val/test là trò chơi zero-sum với tổng số ví dụ cố định.)
- “Test set phải lớn hơn validation set.” (Không nhất thiết; thường gần bằng.)
- “Test set phải lớn hơn cả training set.” (Sai.)

Bài tập: test tốt nhưng ra production tệ

đủ lớn và loss thấp, nhưng mô hình ra thực tế kém. Làm gì? **Đáp án:** xác định dataset khác dữ liệu thực ở đâu. Dataset chỉ là “ảnh chụp” của đời thực; **ground truth** đổi theo thời gian. Có thể cần thu thập lại và retrain/retest.

Bài tập: test set cần bao nhiêu ví dụ?

Đáp án: đủ để kiểm thử có ý nghĩa thống kê. Con số cụ thể phải thử nghiệm theo bài toán, không có phần trăm cố định.

6 Chuyển đổi dữ liệu (Transforming data)

Động cơ học – Tại sao cần khái niệm này?

ô hình ML chỉ train trên float, nhưng nhiều feature không phải float, nên phải biến đổi feature về biểu diễn float.

Tên đường

ả sử
textttstreet names là feature dạng chuỗi như “Broadway”. Mô hình không thể train trên chuỗi, nên phải biến đổi thành số thực (xem module Categorical Data).

Lưu ý

cả feature float cũng thường nên transform (
[hrefhttps://developers.google.com/machine-learning/glossary#normalizationnormalization](https://developers.google.com/machine-learning/glossary#normalizationnormalization))
 để đưa về miền thuận lợi cho huấn luyện (xem module Numerical Data).

6.1 Sample khi dữ liệu quá nhiều**Ý nghĩa & Trực giác**

ếu dataset có quá nhiều example, bạn phải chọn một **tập con** để train. Khi có thể, hãy chọn tập con liên quan nhất tới dự đoán.

6.2 Loại PII**Lưu ý**

tốt nên bỏ ví dụ chứa thông tin định danh cá nhân (PII) để bảo vệ quyền riêng tư; nhưng việc lọc này có thể ảnh hưởng mô hình. (Xem module Safety & Privacy sau.)

7 Tổng quát hóa (Generalization)**Ý nghĩa & Trực giác**

video về một vấn đề phổ biến khi train trên training dataset:

<https://www.youtube.com/watch?v=TyT8i8YIcwI>

8 Quá trình điều chỉnh quá mức (Overfitting)**Ý nghĩa & Trực giác**

Overfitting nghĩa là mô hình khớp (ghi nhớ) tập huấn luyện quá sát đến mức dự đoán kém trên dữ liệu mới. Overfitting rất phổ biến trong ML.

8.1 Ví dụ: rừng cây khỏe / cây bệnh

Hình dưới: kim cương xanh = cây khỏe, vòng tròn cam = cây bệnh (tập huấn luyện).

Thiếu hình minh họa

Không tìm thấy file TreesTrainingSet.png trong thư mục images/overfitting/.

Hình 11. Vị trí cây khỏe/cây bệnh trong tập huấn luyện.

Bạn có thể vẽ đường cong/hình bầu dục để tách hai loại. Một đường phân chia phức tạp có thể phân loại gần như mọi điểm trên train:

Thiếu hình minh họa

Không tìm thấy file `TreesTrainingSetComplexModel.png` trong thư mục `images/overfitting/`.

Hình 12. Mô hình phức tạp trên tập huấn luyện.

Nhưng trên tập kiểm tra (dữ liệu mới), mô hình đó lại sai nhiều:

Thiếu hình minh họa

Không tìm thấy file `TreesTestSetComplexModel.png` trong thư mục `images/overfitting/`.

Hình 13. Cùng mô hình phức tạp trên tập kiểm tra — kém.

8.2 Underfit, fit và overfit

Tình huống	Mô tả
Overfit	Tốt trên train, kém trên dữ liệu mới
Underfit	Kém cả trên train
Fit tốt	Tốt trên train và khái quát tốt

Thiếu hình minh họa

Không tìm thấy file `underfit_fit_overfit.png` trong thư mục `images/overfitting/`.

Hình 14. Underfit, fit, overfit.**Tổng quát hóa**

Generalization là đối lập với overfitting: mô hình dự đoán tốt trên dữ liệu mới. Đây là mục tiêu.

8.3 Phát hiện overfitting

Ý nghĩa & Trực giác

công cụ chính:

- **Loss curve:** loss theo số bước huấn luyện.

- **Generalization curve:** đặt loss train và loss validation lên cùng đồ thị.

Thiếu hình minh họa

Không tìm thấy file `RegularizationTwoLossFunctions.png` trong thư mục `images/overfitting/`.

Hình 15. Hai đường loss tách nhau — gợi ý overfitting.

Lưu ý

đầu hai đường gần nhau; sau đó tách: loss train giảm/ổn định, loss validation tăng $\Rightarrow$ overfitting. Mô hình fit tốt: hai đường có hình dạng tương tự.

8.4 Nguyên nhân overfitting

Tính chất

thường do một hoặc cả hai:

1. Tập huấn luyện không đại diện cho dữ liệu thực (hoặc cho validation/test).
2. Mô hình quá phức tạp so với bài toán.

8.5 Điều kiện tổng quát hóa (điều kiện dataset)

Ba điều kiện thường nhắc

- **IID** (independently and identically distributed): các mẫu độc lập và cùng phân phối.
- **Stationary:** phân phối không đổi mạnh theo thời gian.
- Train/validation/test và dữ liệu thực **cùng phân phối**.

Check your understanding

- Muốn train/val/test giống nhau thống kê? $\rightarrow$ textbfshuffle kỹ trước khi chia; dữ liệu sắp theo thời gian có thể làm phân phối khác nhau.
- Dự đoán độ hot TV 3 năm tới, train trên 10 năm qua, hàng trăm triệu mẫu? $\rightarrow$ có vấn đề vì thị hiếu textbfkhông stationary.
- Dự đoán thời gian đi bộ 1 dặm theo thời tiết trong thành phố 4 mùa? $\rightarrow$ có thể, nếu chia tập sao cho textbfmỗi mùa đều xuất hiện trong train/val/test.

Bài tập thử thách: vé tàu và feedback loop

ô hình gợi ý ngày mua vé tối ưu; loss thấp trên val/test nhưng đôi khi rất tệ ngoài đời.
Lý do có thể là

<https://developers.google.com/machine-learning/glossary>
feedback-loop

textbfeedback loop: gợi ý làm hành vi người dùng đổi, công ty đổi giá, dữ liệu thực thay đổi $\Rightarrow$ mô hình lỗi thời.

9 Độ phức tạp của mô hình (Model complexity) và Regularization

Thiếu hình minh họa

Không tìm thấy file `TreesTestSetComplexModel.png` trong thư mục `images/overfitting/`.

Hình 16. Mô hình phức tạp gây nhiều lỗi trên test set.

Ý nghĩa & Trực giác

ếu thay bằng mô hình đơn giản hơn (đường thẳng) thì sao?

Thiếu hình minh họa

Không tìm thấy file `TreesTestSetSimpleModel.png` trong thư mục `images/overfitting/`.

Hình 17. Mô hình đơn giản (đường thẳng) tổng quát tốt hơn.

Occam's Razor

Ưu tiên đơn giản đã có từ rất lâu; William of Occam hệ thống hóa thành “lưỡi dao Occam”. Nguyên lý này là nền tảng của nhiều khoa học, gồm ML.

Lưu ý

ô hình phức tạp thường tốt hơn trên training set; nhưng mô hình đơn giản thường tốt hơn trên test set (quan trọng hơn).

Bài tập: Occam-friendly hơn?

ông thức vật lý với 3 biến hay 12 biến hợp Occam hơn?**Đáp án: 3 biến.** (Những công thức nổi tiếng như $F = ma$, $E = mc^2$ đều chỉ có 3 biến.)

Bài tập: chọn bao nhiêu feature lúc bắt đầu?

ạn bắt đầu dự án ML mới, sắp chọn feature đầu tiên: chọn bao nhiêu?**Đáp án:** chọn **1–3 feature** có vẻ dự đoán mạnh. Bắt đầu nhỏ giúp xác nhận pipeline hoạt động, dựng baseline nhanh và tránh **curse of dimensionality** (tăng chiều làm không gian phức tạp khiến dữ liệu trở nên thừa, khó học).

9.1 Regularization

Regularization

ô hình cần đồng thời đạt hai mục tiêu mâu thuẫn: (1) khớp dữ liệu tốt; (2) khớp đơn giản nhất có thể. Một cách giữ mô hình đơn giản là phạt mô hình phức tạp trong lúc train — đó là **regularization**.

Ẩn dụ cái còi báo ồn

ưởng tượng mỗi sinh viên có một cái còi làm thầy khó chịu. Mỗi khi bài giảng quá phức tạp, sinh viên bấm còi khiến thầy buộc phải đơn giản hóa. Dần dần, còi “huấn luyện” thầy giảng đủ đơn giản để hiểu, dù có thể kém chi tiết.

9.2 Loss và complexity

Ý nghĩa & Trực giác

ếu chỉ tối ưu $\min(\text{loss})$, mô hình dễ overfit. Tối ưu tốt hơn là $\min(\text{loss} + \text{complexity})$. và **complexity** thường nghịch biến: **complexity** tăng thì **loss** giảm, và ngược lại. Ta cần điểm cân bằng sao cho dự đoán tốt cả trên train lẫn dữ liệu thực.

9.3 Complexity đo bằng gì?

Bài tập: complexity metric hợp lý?

Ý nào hợp lý để đo complexity?

- Complexity là hàm của trọng số (weights). **Có** (L1 regularization).
- Complexity là hàm của bình phương trọng số. **Có** (L2 regularization).
- Complexity là hàm của các bias. **Không** (bias không đo complexity).

10 Điều chỉnh L2 (L_2 regularization) và lambda

Công thức L2 regularization

$$L_2 = w_1^2 + w_2^2 + \dots + w_n^2.$$

Ví dụ tính L2 với 6 trọng số

	Giá trị	Bình phương
w_1	0.2	0.04
w_2	-0.5	0.25
w_3	5.0	25.0
w_4	-1.2	1.44
w_5	0.3	0.09
w_6	-0.1	0.01
		26.83

ận xét: trọng số gần 0 đóng góp ít; trọng số lớn có thể chiếm phần lớn complexity. Trong ví dụ, một mình w_3 đóng góp khoảng 93% tổng.

Lưu ý

2 khuyến khích trọng số tiến về 0 nhưng không đẩy hẳn về 0.

Bài tập: dùng L2 thì complexity thay đổi thế nào?

Đáp án: complexity tổng thể thường giảm vì L2 kéo weights về 0.

Bài tập: L2 có loại bỏ feature không?

Đáp án: Không. Dù có thể làm một số weight rất nhỏ, L2 không đẩy weight nào về đúng 0, nên mọi feature vẫn đóng góp một chút.

10.1 Regularization rate (lambda)

Ý nghĩa & Trực giác

tối ưu min(loss + $\lambda \cdot \text{complexity}$).
(λ regularization rate).

Lambda cao vs thấp

cao:

- Tăng ảnh hưởng regularization, giảm nguy cơ overfit.
- Histogram weights thường gần phân bố chuẩn, mean 0.

thấp:

- Giảm regularization, tăng nguy cơ overfit.
- Histogram weights “phẳng” hơn.

Thiếu hình minh họa

Không tìm thấy file HighLambda.png trong thư mục images/overfitting/.

Hình 18. Histogram weights khi lambda cao (mean 0, gần chuẩn).

Thiếu hình minh họa

Không tìm thấy file LowLambda.png trong thư mục images/overfitting/.

Hình 19. Histogram weights khi lambda thấp (phẳng hơn).

Lưu ý

Đặt $\lambda = 0$ là bỏ regularization hoàn toàn; training chỉ tối thiểu loss $\Rightarrow$ rủi ro overfit cao nhất.

10.2 Chọn lambda

Ý nghĩa & Trực giác

“lý tưởng” phụ thuộc dữ liệu, nên phải tuning.

10.3 Early stopping: lựa chọn thay thế

Early stopping

Early stopping là regularization không cần đo complexity: dừng train trước khi hội tụ hoàn toàn, ví dụ dừng khi loss validation bắt đầu tăng (độ dốc dương). Thường tăng training loss nhưng có thể giảm test loss.

Lưu ý

stopping nhanh nhưng hiếm khi tối ưu; mô hình thường không tốt bằng mô hình được train kỹ với lambda lý tưởng.

10.4 Cân bằng learning rate và regularization rate**Ý nghĩa & Trực giác**

rate và regularization rate thường kéo weights theo hai hướng ngược nhau: learning rate cao kéo weights **ra xa** 0; regularization rate cao đẩy weights **về** 0. ều regularization quá cao so với learning rate: weights yếu $\Rightarrow$ dự đoán kém. Nếu learning rate quá cao so với regularization: weights mạnh $\Rightarrow$ overfit. ục tiêu: tìm điểm cân bằng; nhưng khi đổi learning rate bạn có thể phải tìm lại lambda tối ưu.

11 Diễn giải đường cong tổn thất (Interpreting loss curves)**Ý nghĩa & Trực giác**

sẽ dễ hơn nhiều nếu lần đầu train loss curve luôn như “lý tưởng”.

Thiếu hình minh họa

Không tìm thấy file `metric-curve-ideal.png` trong thư mục `images/overfitting/`.

Hình 20. Loss lý tưởng: giảm nhanh rồi phẳng dần tới minimum.

Lưu ý

ực tế loss curve thường khó đọc. Dưới đây là các bài tập dựa trên trực giác.

11.1 Bài tập 1: loss dao động (oscillating)**Thiếu hình minh họa**

Không tìm thấy file `metric-curve-ex03.png` trong thư mục `images/overfitting/`.

Hình 21. Loss dao động, không phẳng.

Bạn có thể làm 3 việc gì để cải thiện?

- Kiểm dữ liệu theo data schema để phát hiện ví dụ xấu và loại bỏ. (Thực hành tốt cho mọi mô hình.)
- Giảm learning rate. (Thường hữu ích khi debug train.)
- Giảm training set xuống một tập nhỏ, đáng tin để mô hình hội tụ; sau đó thêm dần ví dụ để tìm ví dụ nào gây dao động. (Nghe “giả” nhưng thực sự là kỹ thuật tốt.)

ông nên kỳ vọng tăng số ví dụ hoặc tăng learning rate sẽ sửa dạng curve này.

11.2 Bài tập 2: loss nhảy vọt (sharp jump)**Thiếu hình minh họa**

Không tìm thấy file `metric-curve-ex02.png` trong thư mục `images/overfitting/`.

Hình 22. Loss giảm rồi đột ngột tăng.

Hai nguyên nhân khả dĩ?

- Input có NaN (ví dụ do chia cho 0). (Phổ biến hơn bạn tưởng.)
- Input có một burst outliers (do shuffle batch kém làm một batch chứa rất nhiều outlier).

rate quá thấp không gây hình dạng này; regularization quá cao có thể làm khó hội tụ nhưng không giải thích jump như vậy.

11.3 Bài tập 3: validation loss tăng còn training loss giảm**Thiếu hình minh họa**

Không tìm thấy file `metric-curve-ex01.png` trong thư mục `images/overfitting/`.

Hình 23. Train loss hội tụ nhưng validation loss tăng sau một thời điểm.

Nguyên nhân tốt nhất?

Đáp án: mô hình đang overfit training set. Cách xử lý:

- Làm mô hình đơn giản hơn (giảm số feature).
- Tăng regularization rate.
- Đảm bảo training set và test/validation set tương đương thống kê.

11.4 Bài tập 4: loss bị kẹt / dạng sóng chữ nhật

Thiếu hình minh họa

Không tìm thấy file `metric-curve-ex05.png` trong thư mục `images/overfitting/`.

Hình 24. Loss tạo pattern như sóng chữ nhật.

Giải thích khả dĩ nhất?

Đáp án: training set không được shuffle tốt. Ví dụ: 100 ảnh chó rồi 100 ảnh mèo làm loss dao động; cần shuffle đủ. Regularization quá cao hoặc quá nhiều feature ít khả năng gây pattern này.

12 Tổng kết

Tổng kết chương

- Dữ liệu quyết định sức khỏe mô hình: ưu tiên chất lượng dataset.
- Hiểu reliability: thiếu giá trị, trùng lặp, feature lỗi, nhãn sai, đoạn dữ liệu xấu.
- Xử lý missing: xóa hoặc impute; nếu impute, nên thêm cờ cho biết giá trị bị impute.
- Chia train/validation/test đúng cách; tránh “teaching to the test”; loại bỏ trùng train-test.
- Overfitting: phát hiện qua generalization curve; nguyên nhân chính: dataset không đại diện hoặc mô hình quá phức tạp.
- Regularization: tối ưu loss + complexity; L2 với lambda; early stopping; cân bằng learning rate và lambda.
- Loss curves: dao động, nhảy vọt, tách train/val, pattern do shuffle kém.